

Function F5: $a \cdot b^x$

Deliverable 2 — Problems 5–7

From-scratch implementation, GUI, repository & requirements

Aafreen Fathima

Student ID: 40331369

SOEN 6011 (Section CC) — Summer 2026 — Concordia University

Recap: F5 and the chosen approach

Function. $f(x) = a \cdot b^x$ with $b > 0$. Models exponential growth ($b > 1$) and decay ($0 < b < 1$).

Selected algorithm (D1, Problem 4): $a \cdot b^x = a \cdot \exp(x \cdot \ln b)$

This deliverable (D2):

- Problem 5 — reimplement from scratch (no libraries), add a Tkinter GUI and exception handling.
- Problem 6 — public Git repository with high-quality commit history.
- Problem 7 — update the requirements based on what implementation revealed.

Problem 5 — Implementation from Scratch

Module structure

Module	Responsibility
f5_mathcore.py	From-scratch ln, exp, helpers; power_abx. No libraries.
f5_exceptions.py	CalculatorError → DomainError, OverflowGuard, InputError.
f5_gui.py	Tkinter interface; parses input, calls the core, shows results/errors.
test_f5_d2.py	PyUnit suite (primitives, accuracy, guards).

Run: `python3 f5_gui.py`

Core: the arithmetic primitives (from scratch)

```
def _int_pow2(n):          # 2**n without the ** operator
    negative = n < 0
    if negative: n = -n
    result, base = 1.0, 2.0
    while n > 0:
        if n - (n // 2) * 2 == 1:    # n is odd
            result = result * base
            base = base * base
        n = n // 2                  # exponentiation by squaring
    return 1.0 / result if negative else result
```

Core: computing $a \cdot b^x$

```
def power_abx(a, b, x):
    if b <= 0.0:
        raise DomainError("base b must be > 0 ...")    # REQ-F-4
    if a == 0.0: return 0.0                             # REQ-F-6
    if b == 1.0: return a                               # REQ-F-6
    t = x * _ln(b)    # _ln : atanh series (from scratch)
    m = _exp(t)       # _exp: Taylor + range reduction
    result = a * m
    inf = 1e308 * 10.0
    if result == inf or result == -inf:                  # REQ-R-1
        raise OverflowGuard("result too large ...")
    return result
```

Exception handling & helpful messages

A small hierarchy keeps errors domain-specific instead of leaking raw Python exceptions.

Exception	When & message
DomainError	$b \leq 0$: “The base b must be greater than 0 ... not a real number.”
OverflowGuard	result out of range: “... too large ... Try a smaller x or a base closer to 1.”
InputError	empty/non-numeric field: names the field — “‘abc’ is not a valid number for ‘b’ ...”

Graphical user interface (Tkinter)

$f(x) = a \cdot b^x$

Computes a times b to the power x, from scratch ($b > 0$).

a (coefficient)

b (base, > 0)

x (exponent)

Compute

```
3.0 * 2.0 ^ 4.0
Full precision : 48.0
Fixed (10 dp)  : 48.0000000000
```

- Labelled fields for a, b, x (REQ-U-1)
- Compute / Clear; Enter also computes
- Result in full precision and fixed 10 dp (REQ-P-3)
- Errors appear in-panel, in red, never a crash
- Standard library only: python3 f5_gui.py

Shown: $3 \cdot 2^4 = 48$.

Verification

The from-scratch core is checked against a reference (used only in tests) with a PyUnit suite.

- Primitives: `_abs`, `_floor_to_int`, `_round_to_int`, `_int_pow2` vs. expected.
- Accuracy: 30,000 randomized inputs — worst relative error $\approx 6.5e-14$.
- Spot values: $3 \cdot 2^4 = 48$, $100 \cdot 0.5^3 = 12.5$, $b=1 \rightarrow a$, $a=0 \rightarrow 0$, negative a .
- Guards: negative base, zero base, overflow in exp AND in the final multiply.

```
$ python3 -m unittest test_f5_d2 -v → Ran 10 tests — OK
```

Two bugs found by testing

Testing the from-scratch primitives caught two defects before submission:

- 1. Negative rounding.** `_round_to_int(-2.6)` returned `-4`, not `-3` — the negative branch subtracted twice. Fixed by using `_floor_to_int(v + 0.5)` for both signs.
- 2. False overflow.** An inf test of the form `result == result + 1.0` fired on large finite values. Replaced with a comparison against a computed $\pm\infty$ constant.

Problem 6 — Version Control

Repository & commit quality

Public repository: <https://github.com/aafreen-fathima/soen6011-f5-calculator>

Implement $a*b^x$ numerical core from scratch

Compute $a*b^x = a*\exp(x*\ln b)$ without the math library or the `**` operator. `ln` uses range reduction plus an atanh series; `exp` uses range reduction plus a Taylor series. Re-implement the numeric helpers by hand: `_abs`, `_floor_to_int`, `_round_to_int`, `_int_pow2`. Overflow guarded in `exp` and after the final multiply.

Problem 7 — Updated Requirements

What implementation revealed

Building the function from scratch exposed gaps in the D1 requirements:

An unmet target. REQ-P-2 demanded “ ≥ 15 significant digits.” Measured worst-case error is $\sim 1e-13$ (~ 13 digits) — not achievable with these series.

A changed interface. REQ-I-1 said “textual”; Problem 5 mandates a GUI.

New obligations. Degenerate cases, loop termination, and the from-scratch rule itself had no requirement.

Key requirement changes

ID	Change	Detail
REQ-P-2	revised	“≥15 sig. digits” → “relative error ≤ 1e-12” — verifiable, and met (tested).
REQ-P-3	revised	Two concrete formats: full round-trip precision + fixed 10 dp.
REQ-I-1	revised	Textual UI → graphical UI (Tkinter).
REQ-U-2	revised	Now covers empty fields as well as non-numeric; names the field.
REQ-F-6	NEW	a=0→0; b=1→a, short-circuiting before the series.
REQ-R-3	NEW	Each convergence loop bounded (MAX_ITER) — always terminates.
REQ-C-1	NEW	Core must avoid math, **, and built-in numeric helpers.

References

- [1] ISO/IEC/IEEE 29148:2018 — Requirements engineering. ISO, 2018.
- [2] A. Cooper et al., About Face: The Essentials of Interaction Design, 4th ed. Wiley, 2014.
- [3] W. H. Press et al., Numerical Recipes, 3rd ed. Cambridge Univ. Press, 2007.
- [4] M. Abramowitz and I. A. Stegun, Handbook of Mathematical Functions. Dover, 1965.
- [5] IEEE 754-2019 — Standard for Floating-Point Arithmetic. IEEE, 2019.
- [6] Python Software Foundation, “tkinter” and “unittest,” Python 3 Documentation. <https://docs.python.org/3/>

Summary

- Problem 5 — $a \cdot b^x$ reimplemented from scratch (no libraries, no `**`), with a Tkinter GUI and a custom exception hierarchy. Verified: 10 PyUnit tests, worst error $\sim 6.5e-14$.
- Problem 6 — public Git repository, README, and five scoped, high-quality commits.
- Problem 7 — requirements revised against the implementation: an unmet precision target made testable, interface updated to GUI, three new requirements — with stable identifiers.